

Decision Making Statements:

1. if Statement

The if statement is the simplest decision-making statement. It executes a block of code only if a given condition is true.

Syntax:

```
if(condition)
{
    // Statement;
}
```

Example:

```
class Geeks {
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15) {
            System.out.println("Condition is True");
        }
    }
}
```

2. if-else Statement

The if-else statement allows you to execute one block if the condition is true and another block if it is false.

Syntax:

```
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

Example:

```
import java.util.*;

class Geeks {
```

```

public static void main(String args[])
{
    int i = 10;

    if (i < 15)
        System.out.println("i is smaller than 15");
    else
        System.out.println("i is greater than 15");
}
}

```

3. **nested-if Statement**

A nested-if is an if statement inside another if statement. It is useful when a second condition depends on the first.

Syntax;

```

if (condition1) {
    // code to run if condition1 is true

    if (condition2) {
        // code to run if both condition1 and condition2 are true
    }
}

```

Example:

```

class Geeks {
    public static void main(String args[])
    {
        int i = 10;
        // Outer if statement
        if (i < 15) {
            System.out.println("i is smaller than 15");
            // Nested if statement
            if (i == 10) {

```

```

        System.out.println("i is exactly 10");
    }
}
}
}

```

4. else-if ladder

The if-else-if ladder allows multiple independent conditions to be checked in order. As soon as one condition is true, its block executes, and the rest are skipped.

Syntax:

```

if (condition1) {
    // block of code to be executed if condition1 is true
} else if (condition2) {
    // block of code to be executed if the condition1 is false and condition2 is true
} else {
    // block of code to be executed if the condition1 is false and condition2 is false
}

```

Example:

```

import java.util.*;

class Geeks {
    public static void main(String args[])
    {
        int i = 20;
        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)
            System.out.println("i is 20");
        else
            System.out.println("i is not present");
    }
}

```

```
}
```

5. Switch Case

The switch statement is a multiway branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.

Syntax:

```
switch (expression) {  
  
    case value1:  
  
        // code block for value1  
  
        break;  
  
    case value2:  
  
        // code block for value2  
  
        break;  
  
    // ... more cases  
  
    default:  
  
        // code block if no case matches  
  
}
```

Example:

```
public class DayOfWeek {  
    public static void main(String[] args) {  
        int day = 3;  
        String dayName;  
        switch (day) {  
            case 1:  
                dayName = "Monday";  
                break;  
            case 2:  
                dayName = "Tuesday";  
                break;  
            case 3:  
                dayName = "Wednesday";  
                break;  
            case 4:  
                dayName = "Thursday";  
                break;  
            case 5:  
                dayName = "Friday";
```

```

        break;
    case 6:
        dayName = "Saturday";
        break;
    case 7:
        dayName = "Sunday";
        break;
    default:
        dayName = "Invalid day";
    }
    System.out.println("Today is " + dayName); // Output: Today is Wednesday
}
}

```

Looping statements

Loops can execute a block of code as long as a specified condition is true.

Loops are handy because they save time, reduce errors, and they make code more readable.

1. While Loop

The while loop repeats a block of code as long as the specified condition is **true**:

Syntax:

```

while (condition) {
    // Code to be executed repeatedly
}

```

The condition is evaluated.

If the condition is true, the code block inside the while loop is executed.

After the code block finishes executing, the condition is evaluated again.

This process repeats until the condition evaluates to false.

Once the condition is false, the loop terminates, and the program continues with the statement immediately following the while loop.

Example:

```

public class WhileLoopExample {
    public static void main(String[] args) {
        int count = 1;

        // Loop to print numbers from 1 to 5
        while (count <= 5) {
            System.out.println("Count is: " + count);
        }
    }
}

```

```

        count++; // Increment the counter
    }
    System.out.println("Loop finished.");
}
}

```

2. do-while loop

A do-while loop is a control flow statement in programming that executes a block of code at least once, and then repeatedly executes it as long as a specified condition remains true.

Syntax:

```

do
{
Statements;
}
While(condition);

```

The do/while loop is different: it will always run the code block **at least once**, even if the condition is **false** from the start.

Example:

```

public class DoWhileExample {
    public static void main(String[] args) {
        int count = 1;

        do {
            System.out.println("Count is: " + count);
            count++;
        } while (count <= 5); // Loop continues as long as count is less than or equal to 5
    }
}

```

3. for loop

for loop is a control flow statement that allows code to be executed repeatedly based on a given condition. It is commonly used when the number of iterations is known beforehand.

Syntax:

```

for (initialization; condition; increment/decrement) {
    // code to be executed repeatedly
}

```

- **Initialization:**

This part is executed only once at the beginning of the loop. It typically involves declaring and initializing a loop control variable (e.g., `int i = 0;`).

- **Condition:**

This is a boolean expression evaluated before each iteration. If the condition is true, the loop body is executed. If it's false, the loop terminates.

- **Increment/Decrement:**

This part is executed after each iteration of the loop body. It usually modifies the loop control variable (e.g., `i++`, `i--`, `i += 2`).

Example:

1. Print numbers from 1 to 10

```
public class ForLoopExample {
    public static void main(String[] args) {
        // Loop to print numbers from 1 to 10
        for (int i = 1; i <= 10; i++) {
            System.out.println("Number: " + i);
        }
    }
}
```

2. Sum of first 10 natural numbers

```
class WhileExample2 {
    public static void main(String[] args) {
        int i = 1, sum = 0;
        while (i <= 10) {
            sum = sum + i;
            i++;
        }
        System.out.println("Sum of first 10 numbers = " + sum);
    }
}
```

3. Print even numbers between 1 and 20

```
class WhileExample3 {
    public static void main(String[] args) {
        int i = 2;
        while (i <= 20) {
            System.out.println(i);
            i += 2; // increment by 2 to get even numbers
        }
    }
}
```

Jumping statements

1. **break Statement**

The **break** statement is used to **exit a loop immediately**, even if the loop condition is still true.

Example:

```
class BreakExample {
    public static void main(String[] args) {
        int i = 1;
```

```

while (i <= 10) {
    if (i == 5) {
        System.out.println("Loop stopped at i = " + i);
        break; // exit loop
    }
    System.out.println(i);
    i++;
}
System.out.println("After loop");
}
}

```

Note: When i becomes 5, the break statement stops the loop entirely.

2. continue

The **continue** statement skips the **current iteration** of the loop and jumps to the **next iteration**.

```

class ContinueExample {
    public static void main(String[] args) {
        int i = 0;
        while (i < 10) {
            i++;
            if (i == 5) {
                continue; // skip printing 5
            }
            System.out.println(i);
        }
    }
}

```

Note;

When i == 5, the continue statement skips that iteration — so 5 will not be printed.

Differences between break and continue

break	continue
Used to terminate the loop or switch statement immediately.	Used to skip the current iteration and continue with the next one.
Transfers control outside the loop or switch	Transfers control back to the beginning of the loop for the next iteration.
Loop ends completely , no further iterations occur.	Loop continues , but the remaining code in the current iteration is skipped.
Commonly used to exit early when a certain condition is met.	Commonly used to skip certain values or steps while continuing the loop.
Ex: while(i <= 10) { if(i == 5) break; System.out.println(i); i++; }	while(i <= 10) { i++; if(i == 5) continue; System.out.println(i); }

Label

A label is an identifier followed by a colon (:) that is placed before a loop or block. It allows you to control the flow of nested loops more precisely using break and continue statements.

Syntax:

```
labelName:
while (condition) {
    // loop body
}
```

break labelName; → exits the labeled loop.

continue labelName; → jumps to the next iteration of the labeled loop.

1. Using break with a Label

```
class LabelBreakExample {
    public static void main(String[] args) {
        outerLoop: // label
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (i == 2 && j == 2) {
                    break outerLoop; // exits both loops
                }
                System.out.println(i + " " + j);
            }
        }
        System.out.println("Loop ended.");
    }
}
```

OUTPUT:

1 1

1 2

1 3

2 1

Loop ended.

When `i == 2` and `j == 2`, the `break outerLoop;` statement terminates the outer loop, not just the inner one.

2: Using continue with a Label

```
class LabelContinueExample {  
    public static void main(String[] args) {  
        outer:  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (j == 2) {  
                    continue outer; // skip to next iteration of outer loop  
                }  
                System.out.println(i + " " + j);  
            }  
        }  
    }  
}
```

OUTPUT:

1 1

2 1

3 1

When `j == 2`, the program skips the remaining part of the inner loop and moves to the next iteration of the outer loop.